



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

COURSE CODE: CSE 402

COURSE TITLE: Operating System Lab

EXPERIMENT: Lab 05 - Priority Scheduling

Submitted by:

Name: Riya Mone

Student ID: 23101153

Section: C-1

Semester: 4-1

Submitted to:

Atia Rahman Orthi,

Lecturer,

University of Asia Pacific

Lab Report: Priority Scheduling

Problem Statement

Implement and study Priority CPU Scheduling using the given processes, Arrival Time (AT), Burst Time (BT), and Priority values. Observe the effect of different time quantum values, calculate Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average TAT and Average WT, and represent the scheduling result using a Gantt Chart where applicable.

Objective

The objective of this lab is to understand how priority-based CPU scheduling selects processes and how process priority affects execution order and performance. The experiment also examines the effect of Time Quantum (TQ) values in the provided priority scheduling program and calculates the standard scheduling metrics for each process.

Working Principle of Priority Scheduling

In Priority Scheduling, the CPU is assigned according to process priority. In the first supplied program, the scheduler always selects the non-empty queue with the smallest priority number, so a smaller PR value represents a higher priority. Processes with the same priority are handled through a queue, and unfinished processes are returned to that priority queue after their time quantum expires.

The second supplied program uses a given set of Completion Time values and a Gantt Chart to calculate TAT, WT, Average TAT, and Average WT for another priority-scheduling case.

Advantages of Priority Scheduling

- Important or critical processes can be served before less important processes.
- The scheduling policy can be adjusted by changing process priority values.
- It is useful in systems where different tasks have different levels of urgency.
- When combined with a time quantum, processes of the same priority can share CPU time in a controlled way.

Disadvantages of Priority Scheduling

- Low-priority processes may wait for a long time and may suffer from starvation.
- Choosing suitable priority values can make the scheduling policy more complex.
- A very small time quantum can increase context-switching overhead.
- Priority inversion can occur when a high-priority process indirectly waits for a lower-priority process.

Starvation can be reduced by aging, where the priority of a waiting process is gradually improved over time.

Case 1: Priority Scheduling with TQ = 1 and TQ = 3

The first program uses the following process data. In this case, smaller priority numbers are selected first by the program.

Process	AT	BT	Priority
P1	3	4	2
P2	0	4	0
P3	2	3	1
P4	1	6	1
P5	4	2	3

Source Code - Case 1

```
from collections import deque
process = ["P1", "P2", "P3", "P4", "P5"]
at = [3, 0, 2, 1, 4]
bt = [4, 4, 3, 6, 2]
pr = [2, 0, 1, 1, 3]

def table(title, ct):
    tat = [ct[i]-at[i] for i in range(5)]
    wt = [tat[i]-bt[i] for i in range(5)]

    print("\n", title)
    print("P AT BT PR CT TAT WT")

    for i in range(5):
        print(process[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i])

    print("Avg TAT =", sum(tat)/5)
    print("Avg WT =", sum(wt)/5)

def priority(tq):
    rem = bt.copy()
    ct = [0]*5
    q = {0: deque(), 1: deque(), 2: deque(), 3: deque()}
    added = set()
    time = done = 0

    while done < 5:

        for i in range(5):
            if i not in added and at[i] <= time:
                q[pr[i]].append(i)
                added.add(i)

        if not any(q.values()):
            time += 1
            continue

        p = min(k for k in q if q[k])
        i = q[p].popleft()

        run = min(tq, rem[i])

        for _ in range(run):
            rem[i] -= 1
            time += 1

            for j in range(5):
                if j not in added and at[j] <= time:
                    q[pr[j]].append(j)
                    added.add(j)

        if rem[i] == 0:
            ct[i] = time
            done += 1
        else:
            q[p].append(i)

    table("Priority - TQ " + str(tq), ct)

priority(1)
priority(3)
```

Output - Case 1

```
Priority - TQ 1
P AT BT PR CT TAT WT
P1 3 4 2 17 14 10
P2 0 4 0 4 4 0
P3 2 3 1 10 8 5
P4 1 6 1 13 12 6
P5 4 2 3 19 15 13
Avg TAT = 10.6
Avg WT = 6.8
```

```
Priority - TQ 3
P AT BT PR CT TAT WT
P1 3 4 2 17 14 10
P2 0 4 0 4 4 0
P3 2 3 1 10 8 5
P4 1 6 1 13 12 6
P5 4 2 3 19 15 13
Avg TAT = 10.6
Avg WT = 6.8
```

Result Summary - Case 1

P	AT	BT	PR	CT	TAT	WT
P1	3	4	2	17	14	10
P2	0	4	0	4	4	0
P3	2	3	1	10	8	5
P4	1	6	1	13	12	6
P5	4	2	3	19	15	13

For TQ = 1: Average TAT = 10.6 and Average WT = 6.8.

For TQ = 3: Average TAT = 10.6 and Average WT = 6.8.

For this specific dataset, TQ = 1 and TQ = 3 produce the same CT, TAT, WT, Average TAT, and Average WT. Therefore, neither quantum gives a better numerical result in this experiment.

Comparison of TQ = 1 and TQ = 3

Point	TQ = 1	TQ = 3
Average TAT	10.6	10.6
Average WT	6.8	6.8
Result for this data	Same	Same
General behavior	More frequent rotation/checks	Longer slice before rotation
Context-switch tendency	Usually higher	Usually lower

Case 2: Priority Scheduling with Gantt Chart

The second program uses the same Arrival Time and Burst Time values but a different priority list. The supplied Completion Time values are used to calculate TAT and WT, and the program prints the corresponding Gantt Chart.

Process	AT	BT	Priority	CT	TAT	WT
P1	3	4	2	8	5	1
P2	0	4	6	4	4	0
P3	2	3	1	19	17	14
P4	1	6	2	14	13	7
P5	4	2	3	16	12	10

Source Code - Case 2

```
process = ["P1", "P2", "P3", "P4", "P5"]
at = [3, 0, 2, 1, 4]
bt = [4, 4, 3, 6, 2]
priority = [2, 6, 1, 2, 3]

ct = [8, 4, 19, 14, 16]

n = len(process)

tat = [0] * n
wt = [0] * n

for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]

print("Process AT BT Priority CT TAT WT")

for i in range(n):
    print("{:<8} {:<4} {:<4} {:<9} {:<4} {:<5}".format(
        process[i], at[i], bt[i], priority[i], ct[i], tat[i], wt[i]
    ))

avg_tat = sum(tat) / n
avg_wt = sum(wt) / n

print()
print("Average TAT =", avg_tat)
print("Average WT =", avg_wt)

print()
print("Gantt Chart:")
print("0 P2 4 P1 8 P4 14 P5 16 P3 19")
```

Output - Case 2

```
Process AT BT Priority CT TAT WT
P1      3  4  2      8  5
P2      0  4  6      4  4
P3      2  3  1     19 17
P4      1  6  2     14 13
P5      4  2  3     16 12
```

```
Average TAT = 10.2
```

```
Average WT = 6.4
```

```
Gantt Chart:
```

```
0 P2 4 P1 8 P4 14 P5 16 P3 19
```

Result Summary - Case 2

Average TAT = 10.2

Average WT = 6.4

Gantt Chart: 0 - P2 - 4 - P1 - 8 - P4 - 14 - P5 - 16 - P3 - 19

Discussion

The experiment demonstrates two priority-scheduling cases. In the first program, the scheduler maintains separate queues by priority and always selects the smallest available priority number. The same process set was tested with $TQ = 1$ and $TQ = 3$. Both time quantum values produced identical completion, turnaround, and waiting times for this particular data, with Average TAT = 10.6 and Average WT = 6.8.

The second program calculates scheduling metrics from the supplied Completion Time values and prints a Gantt Chart. For this case, the Average TAT is 10.2 and the Average WT is 6.4. The Gantt Chart shows the execution sequence as P2, P1, P4, P5, and P3.

Priority Scheduling is useful when some processes must receive service before others. However, careful priority management is necessary because low-priority processes can wait too long. Techniques such as aging can improve fairness and reduce starvation.

Conclusion

This lab successfully studied Priority CPU Scheduling using two supplied programs. The required CT, TAT, WT, Average TAT, and Average WT values were obtained, and the effect of different time quantum values was observed. For the first dataset, $TQ = 1$ and $TQ = 3$ gave the same numerical performance. The second case demonstrated the use of a Gantt Chart and produced Average TAT = 10.2 and Average WT = 6.4. Overall, Priority Scheduling provides flexible control over CPU allocation, but starvation and priority-management issues must be considered.